[IPC2]Proyecto_3_2S2026

Carnet 1 – 20220114

Resumen

El proyecto de IPC2 número 3 para el segundo semestre del año 2026, trata de la creación de un software para la Superintendencia de Administración Tributaria que conste de un sistema que funcione como una página/sitio web, permitiendo guardar información de los contribuyentes como lo puede ser el RTU y las facturas, permitiendo también hacer consultas y poder obtener informaciones como lo pueden ser, consultas de contribuyentes donde se pueden visualizar sus facturas emitidas o recibidas.

Resumenes de IVA por fecha y NIT, resúmenes por rango de fechas, gráficas y la posibilidad de poderse imprimir dichos reportes en PDF.

El software utiliza una estructura tipo MVC, basada en RazorPages, aprovechando lo otorgado por el framework ASP.NET Core, como lo es la superclase ControllerBase que se uso para todos los controllers. Utilizando la POO, se creo un software funcional y básico que realiza todo lo solicitado por la SAT.

Palabras clave

Regex: es una secuencia de caracteres que forma un patrón de búsqueda dentro de una cadena de caracteres.
API: conjunto de reglas y protocolos que permite que dos aplicaciones se comuniquen entre sí.
HTPP: conjunto de reglas que permite la comunicación entre un navegador y un servidor.
JSON: formato de texto ligero diseñado para almacenar y transportar datos
NIT: (Número de Identificación Tributaria) es un código único que la SAT asigna a personas individuales y jurídicas.

Abstract

"The project number 3 of IPC2 for the second semester of the year 2026 deals with the creation of software for the Superintendency of Tax Administration consisting of a system that functions as a web page/site, allowing to save taxpayer information such as the RTU and invoices, also allowing to make queries and be able to obtain information such as taxpayer queries where issued or received invoices can be visualized. IVA summaries by date and NIT, summaries by date range, charts, and the possibility of being able to print said reports in PDF. The software uses an MVC-type structure, based on RazorPages, taking advantage of what is granted by the ASP.NET Core framework, like the ControllerBase superclass that was used for all controllers. Using OOP, a functional and basic software was created that accomplishes everything requested by the SAT."

Keywords

Regex:It is a sequence of characters that forms a search pattern within a string of characters.

API:Set of rules and protocols that allows two applications to communicate with each other.

HTTP:Set of rules that allows communication between a browser and a server.

JSON:Lightweight text format designed to store and transport data.

NIT:(Tax Identification Number) is a unique code assigned by the SAT to individuals and legal entities.

Introducción

El software sirve para la SAT, una agencia que se encarga de todos los tramites que incluyen facturación, todos los considerados contribuyentes que pueden ser personas individuales o entidades jurídicas, están identificados dentro de la SAT por medio del RTU y sus NITs, los NITs son códigos que poseen todos los contribuyentes y sirve para poder realizar los distintos tramites que necesiten realizar, la facturación siempre lleva los NITs cuando se realiza. Este software busca permitir obtener las facturas realizadas por los contribuyentes, verificando antes que existan esto por medio de un protocolo que verifica si los NITs ingresados existen o no, y claro pudiendo crear nuevos de ser el caso.

El software por su parte utiliza una arquitectura cliente-servidor, teniendo una separación entre el front end y el backend, utilizando ASP.NET Core Razor Pages, y una API HTTP. Debido a la organización de la API, esta sigue el patrón MVC.

Desarrollo del tema

El software esta pensado realizarse para ofrecer servicios que solicita la Superintendencia de Administración Tributaria (SAT). Dicho software requiere del manejo de ciertas estructuras complejas, para poder ingresar datos, se utiliza a diferencia de proyectos anteriores, archivos de tipo JSON para manejar la información al mismo tiempo que se requiere de un software que actúe como un sitio web, con servidor, manejando en este caso una arquitectura cliente-servidor, se opto por crearse el programa presentado.

El proyecto se divide en 2 partes principales, el backend y el frontend, cada parte con sus clases y diferentes archivos, forman lo que en conjunto es un gran programa que proporciona y cumple con lo solicitado, en el apartado services se valida por medio de regex las entradas, los modelos creados a su vez en la API, trabajan en conjunto con los controllers a su vez para el manejo de la lógica, mientras que en este caso las vistas, son manejadas gracias a las RazorPages.

Las RazorPages son una página con 2 archivos que funcionan en conjunto para poder proveer las vistas que el programa maneja. En conjunto con el backend permite crear un programa independiente, respetando la separación de funcionalidad, esto permite que se pueda modificar/alterar un componente sin que esto afecte al resto.
Todo esto es posible gracias al framework ASP.NET core, el cual proporciona desde las super clases ControllerBase, hasta las RazorPages. Siendo un framework muy completo, es el que nos permitió desarrollar el proyecto tal y como esta implementado, permitiendo ahorrar mucha lógica y código al parshear desde lo que podría ser una más complicada lógica MVC, o el estructurar la lógica HTML más pura, ASP.NET core nos permite concentrarnos en lo más importante al tener código ya echo y poder enfocarnos puramente en la lógica del negocio como lo puede ser, el validar los NITs, guardar los datos de forma permanente y poder incluso imprimir documentos en PDF.

Conclusiones

El trabajo realizado permite ver la enorme e increíble importancia y ventaja que es el usar frameworks que nos permitan ahorrar trabajo y tiempo, al haber frameworks como ASP.NET Core que inclusive nos permite y facilita la elaboración de páginas web y lógica con estructura cliente-servidor, nos permite elaborar un mejor patrón MVC y nos otorga las RazorPages, verdaderamente muestra la utilidad de los frameworks.

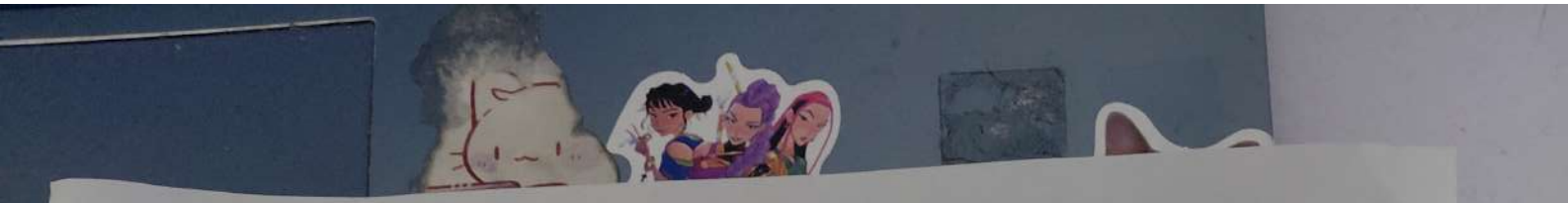
A su vez, las estructuras propias de un lenguaje como lo son Listas o TDAs como DateTime son ejemplos de como tanto un frameowork como un lenguaje de programación pueden ser bastante

Diagrama de Clases

Contribuyente
+String NIT +string NOmbre
+ get NIT +set NIT +get Nombre +Set Nombre

Factura Solicitud
+String TIEMPO +String REFERENCIA +String NIT_EMISOR +String NIT_RECEPTOR +decimal valor +decimal IVA +decimal TOTAL
+getters +setters

Factura Registro
+String TIEMPO Original +String Referencia +string NitEmisor +String Nit Receptor + decimal VAlor + decimal IvaRecibido + decimal Iva Calculado + decimal Total Recibido +decimal Total calculado + DateTime Fecha Factura + DateTime Fecha Procesamiento +bool Aprobada + String codigoAutorizacion + List<String> Errores
+Getters +setters



ResultadoAutorizaciones

+int CantidadAutorizacionesAProbadas
+int CantidadAutorizacionesRechazadasPorNitEmisorInexistente
+int cantidadAutorizacionesRechazadasforNitReceptorInexistente
+int CantidadAutorizacionesRechazadasPorNitEmisorInvalido
+int cantidadAutorizacionesRechazadasPorNitReceptorInvalido
+int CantidadAutorizacionesRechazadasporIvaMalCalculado
+int CantidadAutorizacionesRechazadasPorTotalMalCalculado
+int cantidadAutorizacionesRechazadasPorReferenciaDuplicada
+int cantidadDescartadasPorFormato
+ List<DetalleAProbacionz detalleAProbaciones

+Getters
+setters

DetalleAProbacion

+String referencia
+long codigoAprobacion

+Getters
+setters

RespuestaAutorizaciones

+ ResultadoAutorizaciones resultadoAutorizaciones

+get
+set

RespuestaMtu

+int CONTRIBUYENTES_NUEVOS
+int CONTRIBUYENTES_ACTUALIZADOS
+int NITS_INVALIDOS

+Getters
+setters